A Quantum Approach to Option Pricing

Yash Chauhan[a)]
Roll Number: 2211145, Course Code: P346,
School of Physical Sciences,
National Institute of Science Education and Research,
An OCC of Homi Bhabha National Institute,
Jatni, Bhubaneswar, 752050, Odisha, India

(Dated: 7 December 2024)

Synopsis

This report focuses on reproducing the results of the seminal paper "Option Pricing Using Quantum Computers" by Stamatopoulos *et al.* (2020), which demonstrates the use of quantum amplitude estimation for pricing financial options. By leveraging quantum computing's potential quadratic speed-up over classical Monte Carlo methods, the original work explored pricing vanilla, basket, and path-dependent options on a gate-based quantum computer. In this study, the methodology, quantum circuits, and algorithms presented in the original paper are replicated and analyzed in detail. Additionally, extensions and practical considerations for implementation on modern quantum hardware are discussed, with an emphasis on understanding the challenges and limitations of applying quantum methods to real-world financial problems. This work serves as a hands-on exploration of quantum algorithms in finance and provides insights into their applicability for solving computationally intensive problems in option pricing.

Keywords: Quantum Computing, Option Pricing, Amplitude Estimation, Quantum Finance

---

I. INTRODUCTION

The pricing of financial derivatives, particularly options, represents a cornerstone of modern financial markets. Options offer the holder the right, but not the obligation, to buy or sell an asset at a predetermined price within a specific timeframe. These instruments, whether standard or exotic, serve diverse purposes, from speculative trading to sophisticated hedging strategies. Understanding their valuation is vital for financial engineers, risk managers, and traders alike. Traditional methods for option pricing, such as the Black-Scholes model, provide analytical solutions for basic scenarios. However, their simplifying assumptionslike constant volatility and idealized market behaviormake them inadequate for capturing the complexities of real-world financial instruments (Hull, 2006). In practice, numerical methods, particularly Monte Carlo simulations, are widely adopted for their flexibility in handling stochastic parameters (Boyle, 1977). Despite their strengths, these simulations often demand significant computational resources, particularly for high-dimensional or path-dependent options. This computational bottleneck motivates the exploration of novel technologies to improve efficiency.

Quantum computing offers an innovative approach to tackling computationally intensive problems. Leveraging the principles of superposition and entanglement, quantum algorithms promise to outperform their classical counterparts in specific domains. Finance, with its array of complex problems, stands to benefit from this paradigm shift. Recent developments in quantum algorithms have highlighted their potential applications in portfolio optimization, risk analysis, and derivative pricing. Among these, Amplitude Estimation

---

[a)]Electronic mail: yash.chauhan@niser.ac.in

(AE) has emerged as a key technique, offering a theoretical quadratic speed-up (Brassard *et al.*, 2002) for problems traditionally reliant on Monte Carlo methods.

In this report, I aim to replicate and analyze the findings of a seminal paper on quantum option pricing Stamatopoulos *et al.* (2020). The study explores the implementation of quantum algorithms for pricing various types of options, with an emphasis on practical realizations using gate-based quantum hardware. This work also reflects the potential for quantum computing to address the computational challenges inherent in modern finance. The structure of this report mirrors the paper under study but incorporates a fresh perspective. The methodology section elaborates on the quantum techniques used, including the representation of probability distributions and the application of AE. Subsequently, results from the reproduced experiments are analyzed, highlighting both the strengths and limitations of current quantum hardware. Through this effort, I aim to demonstrate the feasibility of quantum computing for option pricing and its potential impact on the financial industry.

II. TYPES OF OPTIONS

Options can be categorized based on their path dependency and the number of underlying assets. Table I provides an overview of the classifications:

	Single-asset	**Multi-asset**
Path-independent	European Put/Call	Basket Option
Path-dependent	Barrier & Asian Options	Multi-asset Barrier Options

TABLE I. Classification of option types.

Below, we describe the mathematical framework for each type of option.

A. European Put/Call Options

European options are path-independent and can only be exercised at maturity. Their payoff functions are:

$$\text{Call Option: } P_{\text{call}} = \max(S_T - K, 0), \tag{1}$$

$$\text{Put Option: } P_{\text{put}} = \max(K - S_T, 0), \tag{2}$$

where, S_T is the price of the underlying asset at time T and K is the strike price.

B. Basket Options

Basket options depend on a weighted basket of multiple underlying assets. The value of the basket at time T is defined as:

$$B_T = \sum_{i=1}^{n} w_i S_{i,T},$$

where, $S_{i,T}$ is the price of the i-th asset at time T and w_i are the weights assigned to each asset, satisfying $\sum_{i=1}^{n} w_i = 1$.

The payoff for a basket call option is:

$$P_{\text{basket}} = \max(B_T - K, 0). \tag{3}$$

C. Barrier Options

Barrier options are path-dependent and activated or deactivated when the underlying asset reaches a specific barrier B. For a knock-in barrier call option, the payoff is:

$$P_{\text{barrier}} = \begin{cases} \max(S_T - K, 0), & \text{if } \exists t \in [0, T] \text{ such that } S_t \geq B, \\ 0, & \text{otherwise.} \end{cases} \tag{4}$$

D. Asian Options

Asian options depend on the average price of the underlying asset over a predefined period. For an arithmetic average Asian call option, the payoff is:

$$P_{\text{asian}} = \max\left(\frac{1}{n}\sum_{i=1}^{n} S_i - K, 0\right), \tag{5}$$

where, S_i is the price of the asset at the i-th time step and n is the total number of time steps.

E. Multi-Asset Barrier Options

Multi-asset barrier options depend on the prices of multiple underlying assets and incorporate barrier features. For a multi-asset knock-in barrier call, the payoff can be expressed as:

$$P_{\text{multi-barrier}} = \begin{cases} \max\left(\sum_{i=1}^{n} w_i S_{i,T} - K, 0\right), & \text{if } \exists t \in [0, T],\ S_{i,t} \geq B_i \text{ for all } i, \\ 0, & \text{otherwise.} \end{cases} \tag{6}$$

Classifying options based on their features and payoffs provides a framework for understanding their valuation and applications. Each type of option caters to specific market needs, such as managing risk, hedging, or speculating on market movements.

III. GENERAL METHODOLOGY

Option pricing methodologies can broadly be divided into classical and quantum approaches. Both methods aim to estimate the expected payoff of an option, but they differ significantly in how they model stochastic processes and compute the expected value. Below, I describe both approaches in detail.

A. Classical Methodology

The classical method for option pricing relies on Monte Carlo simulation, a flexible numerical technique widely used for handling the stochastic nature of financial markets. The key steps involved are as follows:

1. **Modeling the Stochastic Process:** The underlying asset price is modeled as a set of random variables $\mathbf{X} = \{X_1, X_2, \ldots, X_N\}$ that evolve according to a stochastic process. Typically, the Geometric Brownian Motion (GBM) model is used:

   $$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t,$$

   where S_t is the asset price at time t, μ is the drift, σ is the volatility, and W_t is a Wiener process.

2. **Generating Random Paths:** A total of M independent random paths $\{\mathbf{X}_1, \mathbf{X}_2, \ldots, \mathbf{X}_M\}$ are generated from the probability distribution of the underlying stochastic process. Each path represents a possible trajectory of the asset price.

3. **Computing the Payoff:** For each random path $\mathbf{X}_i$, the option's payoff is computed using a payoff function $f(\mathbf{X}_i)$. For example, the payoff of a European call option is:

$$f(\mathbf{X}_i) = \max(S_T - K, 0),$$

where S_T is the terminal price of the asset and K is the strike price.

4. **Estimating the Expected Payoff:** The expected payoff is estimated by averaging the payoffs over all M random paths:

$$\hat{\mathbb{E}}_{\mathbb{P}}[f(\mathbf{X})] = \frac{1}{M} \sum_{i=1}^{M} f(\mathbf{X}_i),$$

where $\mathbb{P}$ represents the risk-neutral probability measure. The estimator $\hat{\mathbb{E}}_{\mathbb{P}}[f(\mathbf{X})]$ converges to the true expected value $\mathbb{E}_{\mathbb{P}}[f(\mathbf{X})]$ as $M \to \infty$, with a convergence rate of $O(M^{-1/2})$.

While Monte Carlo methods are highly flexible and widely used, their slow convergence rate can lead to significant computational costs, especially for complex options or high-dimensional problems.

B. Quantum Methodology

The quantum approach to option pricing leverages the principles of quantum mechanics to achieve a quadratic speedup over classical Monte Carlo methods. The process can be outlined as follows:

1. **Representing the Probability Distribution:** The probability distribution $\mathbb{P}$ governing the evolution of the random variables $\mathbf{X} = \{X_1, X_2, \ldots, X_N\}$ is encoded onto a quantum computer. This involves preparing a quantum state that corresponds to the probability distribution of the stochastic process.

2. **Constructing the Payoff Circuit:** A quantum circuit is constructed to compute the payoff function $f(\mathbf{X})$. The circuit must efficiently map the quantum representation of $\mathbf{X}$ to a quantum representation of the payoff.

3. **Estimating the Expected Payoff:** The expected payoff $\mathbb{E}_{\mathbb{P}}[f(\mathbf{X})]$ is calculated using a quantum amplitude estimation algorithm. This algorithm provides a quadratic improvement in convergence rate, achieving $O(M^{-1})$ with M quantum samples. This speedup is particularly advantageous for high-dimensional and computationally intensive problems (Stamatopoulos *et al.*, 2020).

The quantum methodology represents a paradigm shift in option pricing by providing the potential for significant computational savings, making it an attractive alternative to classical methods in the area of quantitative finance.

IV. QUANTUM ALGORITHMS FOR OPTION PRICING

Quantum algorithms play a critical role in the implementation of efficient option pricing methodologies. This section delves into the core algorithms used in quantum finance applications, including Amplitude Estimation, Quantum Phase Estimation, and Quantum Iterative Phase Estimation. These algorithms enable the quadratic speedup over classical methods by leveraging quantum mechanics principles.

A. Amplitude Estimation

Amplitude Estimation (AE) is a fundamental quantum algorithm that enables the efficient computation of the expected value of a function encoded in a quantum state (Montanaro, 2015). Unlike classical Monte Carlo methods, which converge at a rate of $O(M^{-1/2})$, AE achieves a convergence rate of $O(M^{-1})$, providing a theoretical quadratic speedup.

Algorithm Framework

The goal of AE is to estimate the amplitude a of a quantum state prepared as:

$$|\psi\rangle = \sqrt{a}\,|1\rangle\,|\phi_1\rangle + \sqrt{1-a}\,|0\rangle\,|\phi_0\rangle\,,$$

where a represents the probability of measuring $|1\rangle$ in the first qubit, and $|\phi_1\rangle$, $|\phi_0\rangle$ are ancillary states.

The key steps of the algorithm are:

1. Define a unitary operator $\mathcal{A}$ that prepares the state $|\psi\rangle$:

$$A\,|0\rangle = \sqrt{a}\,|1\rangle\,|\phi_1\rangle + \sqrt{1-a}\,|0\rangle\,|\phi_0\rangle\,. \tag{7}$$

2. Construct the Grover operator $\mathcal{Q} = (2\,|\psi\rangle\,\langle\psi| - I)\mathcal{A}^\dagger(2\,|0\rangle\,\langle 0| - I)\mathcal{A}$, which performs rotations in the subspace spanned by $|\psi\rangle$ and its orthogonal complement.

3. Apply Quantum Phase Estimation (QPE) to estimate the eigenvalue $e^{2\pi i\theta}$ of $\mathcal{Q}$, where θ is related to the amplitude a by:

$$a = \sin^2(\pi\theta).$$

4. Compute the amplitude a from the estimated θ value. The estimator $\tilde{a}$ for a is given by:

$$\tilde{a} = \sin^2\left(\frac{y\pi}{M}\right) \in [0,1]$$

   where $y \in \{0,\ldots,M-1\}$ and $M = 2^m$ is the number of quantum samples. The m qubits are initialized to an equal superposition state by Hadamard gates, and are used to control different powers of $\mathcal{Q}$.

The probability amplitude a is derived from the eigenvalue decomposition of $\mathcal{Q}$:

$$\mathcal{Q}\,|\psi_\pm\rangle = e^{\pm 2\pi i\theta}\,|\psi_\pm\rangle\,,$$

where $|\psi_\pm\rangle$ are the eigenstates of $\mathcal{Q}$. By measuring the phase θ using QPE, the amplitude a can be efficiently estimated. The amplitude estimator a converges quadratically as $O(M^{-1})$:

$$|a - \tilde{a}| \leq \frac{\pi}{M} + \frac{\pi^2}{M^2} = O\left(\frac{1}{M}\right) \tag{8}$$

B. Quantum Phase Estimation (QPE)

Quantum Phase Estimation is a foundational quantum algorithm used to estimate the eigenvalue (phase) of a unitary operator. It forms the backbone of Amplitude Estimation and other quantum algorithms (Nielsen and Chuang, 2000).

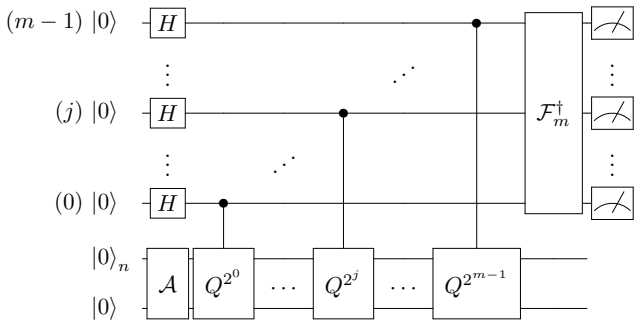


FIG. 1. The quantum circuit of amplitude estimation, where H denotes a Hadamard gate and $\mathcal{F}^\dagger$ the inverse QFT.

Algorithm Framework

Given a unitary operator U and an eigenstate $|\psi\rangle$ such that:

$$U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle,$$

the goal is to estimate the phase $\phi \in [0, 1)$.

The algorithm proceeds as follows:

1. Prepare an initial state:

$$|\psi_0\rangle = \frac{1}{\sqrt{2^t}} \sum_{k=0}^{2^t-1} |k\rangle \otimes |\psi\rangle,$$

   where t is the number of qubits in the first register.

2. Apply controlled-U^{2^j} gates to the second register, conditioned on the qubits of the first register:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^t}} \sum_{k=0}^{2^t-1} |k\rangle \otimes U^k|\psi\rangle.$$

3. Perform an inverse Quantum Fourier Transform (QFT) on the first register to extract the phase ϕ.

4. Measure the first register to obtain the binary representation of ϕ.

The state after applying controlled-U gates can be expressed as:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^t}} \sum_{k=0}^{2^t-1} e^{2\pi ik\phi}|k\rangle \otimes |\psi\rangle.$$

After applying the inverse QFT, the first register collapses to a state representing the phase ϕ with high probability.

C. Iterative Amplitude Estimation

To reduce qubit requirements and circuit depth, Suzuki *et al.* (2020) demonstrated that AE can achieve quadratic speed-up without QPE. This is done by measuring $\mathcal{Q}^k\mathcal{A}|0\rangle$ for

$k = 2^0, \ldots, 2^{m-1}$ and using maximum likelihood estimation to approximate θ_a, and thus a. The method exploits $\mathcal{Q}^k\mathcal{A}\left|0\right\rangle_n\left|0\right\rangle = \cos\left((2k+1)\theta_a\right)\left|\psi_0\right\rangle_n\left|0\right\rangle + \sin\left((2k+1)\theta_a\right)\left|\psi_1\right\rangle_n\left|1\right\rangle$, where $M = 2^m - 1$ applications of $\mathcal{Q}$ with N shots yield an estimation error scaling as $\mathcal{O}(1/(M\sqrt{N}))$, retaining the speed-up.

V. QUANTUM METHODOLOGY

In this section, we detail the quantum methods utilized for option pricing, building upon the Amplitude Estimation (AE) algorithm introduced earlier. The methodology involves encoding probability distributions onto a quantum computer, applying the operator $\mathcal{A}$ to represent payoffs, and efficiently estimating expectations. We outline the three key steps in the quantum methodology: leveraging Amplitude Estimation for efficient convergence, loading distributions into a quantum state, and computing the payoff function.

A. Amplitude Estimation for Option Pricing

As discussed in the previous section, Amplitude Estimation (AE) provides a quadratic speedup for estimating expectations compared to classical Monte Carlo methods. In the context of option pricing, the operator $\mathcal{A}$ represents the payoff function, mapping asset price probabilities to their corresponding payoffs. Specifically, $\mathcal{A}$ is defined as:

$$\mathcal{A}\left|0\right\rangle_{n+1} = \sum_{i=0}^{2^n-1}\sqrt{p_i}\left|i\right\rangle_n\left(\sqrt{1-f(S_i)}\left|0\right\rangle + \sqrt{f(S_i)}\left|1\right\rangle\right), \tag{9}$$

where S_i are discretized asset prices, p_i their associated probabilities, and $f(S_i)$ the payoff function.

AE iteratively applies the Grover operator:

$$\mathcal{Q} = \mathcal{A}S_0\mathcal{A}^\dagger S_{\psi_0}, \tag{10}$$

where $S_0 = 1 - 2\left|0\right\rangle\left\langle 0\right|$, $S_{\psi_0} = 1 - 2\left|\psi_0\right\rangle\left|0\right\rangle\left\langle\psi_0\right|\left\langle 0\right|$, to amplify the desired amplitudes. The expected payoff is recovered as $\mathbb{E}[f(S)]$ by estimating the probability amplitude $a = \sin^2(\theta_a)$. This quadratic convergence reduces the computational complexity of estimating option prices.

B. Distribution Loading and Uncertainty Modeling

Modeling real-world financial data is challenging due to market complexities and noise. Quantum Generative Adversarial Networks (qGANs) have been proposed to model such data efficiently, leveraging the ability to learn complex distributions with fewer parameters (Zoufal, Lucchi, and Woerner, 2019). However, for simplicity and computational feasibility, we load a log-normal distribution, which is commonly used in financial modeling to approximate stock price dynamics.

The log-normal distribution is truncated to an interval $[\text{low}, \text{high}]$ and discretized into 2^n grid points, where n is the number of qubits. This process is represented by the transformation:

$$\left|0\right\rangle_n \to \left|\psi\right\rangle_n = \sum_{i=0}^{2^n-1}\sqrt{p_i}\left|i\right\rangle_n,$$

where p_i corresponds to the probabilities of the discretized distribution. The index i is mapped to the interval using the affine transformation:

$$i \mapsto \frac{\text{high} - \text{low}}{2^n - 1}\cdot i + \text{low}.$$

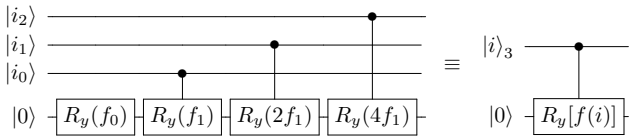


FIG. 2. Quantum circuit for creating the state in Eq. (12). The variable $i = 4i_2 + 2i_1 + i_0 \in \{0, \ldots, 7\}$ is encoded by three qubits as $|i\rangle_3 = |i_2 i_1 i_0\rangle$, with $f(i) = f_1 i + f_0$ expressed as $4f_1 i_2 + 2f_1 i_1 + f_1 i_0 + f_0$. The resulting state is $|i\rangle_3 [\cos(f_1 i + f_0)\,|0\rangle + \sin(f_1 i + f_0)\,|1\rangle]$. The circuit on the right shows a simplified notation.

This ensures a robust representation of the underlying asset price dynamics in the quantum state (Grover and Rudolph, 2002).

C. Computing the Payoff

Once the probability distribution is encoded, the payoff function $f(S)$ is implemented via controlled rotations. For options, the payoff function is often piecewise linear, enabling efficient encoding using a linear transformation:

$$f(i) = f_1 i + f_0, \tag{11}$$

where f_1 and f_0 are constants derived from the option contract's parameters. The state prepared by $\mathcal{A}$ is:

$$\mathcal{A}\,|0\rangle = \sum_{i=0}^{2^n-1} \sqrt{p_i}\,|i\rangle \left(\cos(f(i))\,|0\rangle + \sin(f(i))\,|1\rangle\right). \tag{12}$$

This maps the probabilities p_i to the payoffs $f(i)$, allowing for efficient expectation value estimation using AE.

To scale the payoff function within the range $[-1, 1]$, we redefine it as:

$$\tilde{f}(i) = 2\frac{f(i) - f_{\min}}{f_{\max} - f_{\min}} - 1, \tag{13}$$

where $f_{\min}$ and $f_{\max}$ are the minimum and maximum payoffs, respectively. The ancilla qubit's state encodes the scaled payoff:

$$\cos\left(c\tilde{f}(i) + \frac{\pi}{4}\right)|0\rangle + \sin\left(c\tilde{f}(i) + \frac{\pi}{4}\right)|1\rangle, \tag{14}$$

where $c \in [0, 1]$ is a scaling factor. The probability of measuring the ancilla in state $|1\rangle$ is:

$$P_1 = \sum_{i=0}^{2^n-1} p_i \sin^2\left(c\tilde{f}(i) + \frac{\pi}{4}\right). \tag{15}$$

For small $c\tilde{f}(i)$, we approximate:

$$P_1 \approx c\frac{2\mathbb{E}[f(S)] - f_{\min}}{f_{\max} - f_{\min}} - c + \frac{1}{2}. \tag{16}$$

This provides a direct link between the measured probability P_1 and the option's expected payoff $\mathbb{E}[f(S)]$, completing the quantum pricing methodology.

VI. OPTION PRICING ON A QUANTUM COMPUTER

In this section, we outline how to price different types of options, focusing on the implementation of quantum circuits required for Amplitude Estimation (AE). These circuits encode the necessary probability distributions and payoff functions into quantum states, as described in the methodology section.

A. Path-Independent Options

Path-independent options, such as vanilla European call and put options, depend solely on the distribution of the underlying asset price at maturity, S_T, and the payoff function $f(S_T)$. To represent the distribution of S_T on a quantum computer, we truncate it to the range $[S_{T,\min}, S_{T,\max}]$ and discretize this range into 2^n intervals using n qubits as described in Section V B. Using this state, we first demonstrate the quantum pricing of vanilla options and extend the method to handle portfolios of multiple vanilla options.

1. Vanilla Options

Vanilla options have simple payoff structures. For a call option, the payoff function is:

$$f_C(S_T) = \max(0, S_T - K), \tag{17}$$

where K is the strike price. For a put option, the payoff is given by:

$$f_P(S_T) = \max(0, K - S_T). \tag{18}$$

The payoff function for call options pays the excess of the asset price over the strike K, while put options pay the excess of K over the asset price. These payoffs involve a max operation, which needs to be implemented as a quantum circuit.

To implement the max operation, we use a quantum comparator circuit. This circuit compares the asset price i encoded in the quantum state with the strike price K. An ancillary qubit $|c\rangle$, initialized to $|0\rangle$, is flipped to $|1\rangle$ if $i \geq K$. This transformation is expressed as:

$$|\psi\rangle_n |0\rangle \to \sum_{i<K} \sqrt{p_i} |i\rangle_n |0\rangle + \sum_{i\geq K} \sqrt{p_i} |i\rangle_n |1\rangle . \tag{19}$$

The comparator circuit uses Toffoli and CNOT gates to calculate the twos complement of K and compare it with the values encoded in the quantum state. Based on the result, an ancilla qubit is flipped to indicate whether the condition $i \geq K$ holds. A specific implementation of this circuit for three qubits is shown in Fig. 3.

After applying the comparator circuit, we encode the payoff function as a controlled rotation on another ancilla qubit. For a European call option, the payoff is $f(i) = \max(0, i - K)$, which is piecewise linear:

$$f(i) = \begin{cases} 0 & \text{if } i < K, \\ i - K & \text{if } i \geq K. \end{cases} \tag{20}$$

To encode this payoff, we perform a controlled rotation on the ancilla qubit using angles $g(i)$ and g_0 that depend on i and K:

$$g(i) = \frac{2c(i-K)}{i_{\max} - K}, \quad g_0 = \frac{\pi}{4} - c, \tag{21}$$

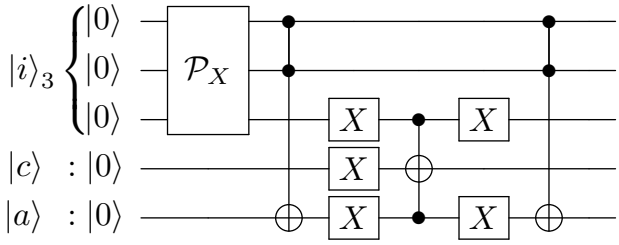


FIG. 3. Quantum circuit that sets the comparator qubit $|c\rangle$ to $|1\rangle$ if $|i\rangle_3$ exceeds strike K. The unitary $\mathcal{P}_X$ loads the probability distribution. Another ancilla qubit $|a\rangle$ is needed, which is uncomputed after the comparison.

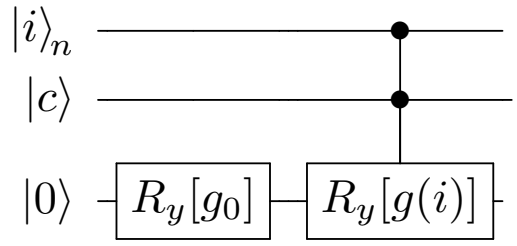


FIG. 4. Circuit that creates the state in Eq. (24). We apply this circuit directly after the comparator circuit shown in Fig. 3. The multi-controlled y-rotation is the gate shown in Fig. 2 controlled by the ancilla qubit $|c\rangle$ that contains the result of the comparison between i and K.

where $i_{\max} = 2^n - 1$ is the maximum encoded value. These angles ensure that the scaled payoff $c\tilde{f}(i)$ is within $[-c, c]$, enabling efficient estimation of the expected payoff using AE.

The final quantum state, after applying the rotations, is:

$$\sum_{i<K} \sqrt{p_i}\,|i\rangle_n\,|0\rangle\,[\cos(g_0)\,|0\rangle + \sin(g_0)\,|1\rangle] + \sum_{i\geq K} \sqrt{p_i}\,|i\rangle_n\,|1\rangle\,[\cos(g_0+g(i))\,|0\rangle + \sin(g_0+g(i))\,|1\rangle]\,. \tag{22}$$

The probability of measuring the ancilla in state $|1\rangle$ is:

$$P_1 = \sum_{i<K} p_i \sin^2(g_0) + \sum_{i\geq K} p_i \sin^2[g_0 + g(i)]. \tag{23}$$

From P_1, we can retrieve the expected payoff $\mathbb{E}[f(S_T)]$ using a scaling factor. An operator can be constructed to act as

$$|x\rangle\,|0\rangle \to |x\rangle\,(\cos(a\cdot x + b)\,|0\rangle + \sin(a\cdot x + b)\,|1\rangle)\,, \tag{24}$$

using controlled Y-rotations.

The same method applies to put options, where the payoff function is $f(i) = \max(0, K - i)$. The circuit design and angles are adjusted accordingly.

B. Path-Dependent Options

Path-dependent options are characterized by payoffs that depend on the price of the underlying asset at multiple time intervals rather than just at maturity. Two commonly

studied examples are Asian options and barrier options. Below, we describe the quantum pricing methodology for each of these.

1. Asian Options

Asian options, also known as average-price options, are path-dependent options where the payoff is determined by the average price of the underlying asset over a specified time period. For an Asian call option, the payoff is given by:

$$f(\bar{S}) = \max(0, \bar{S} - K),$$

where K is the strike price, and $\bar{S}$ represents the arithmetic average of the asset prices over d time steps:

$$\bar{S} = \frac{1}{d} \sum_{t=1}^{d} S_t.$$

On a quantum computer, the asset price at each time step is represented using a quantum register. The multivariate probability distribution of the asset prices at different time intervals is loaded into these registers. For simplicity, we assume the prices follow a log-normal distribution with the probability density function:

$$P(S_t) = \frac{1}{S_t \sigma \sqrt{2\pi \Delta t}} e^{-\frac{(\ln S_t - \mu_t)^2}{2\sigma^2 \Delta t}},$$

where $\Delta t = T/d$ is the time interval between steps, and $\mu_t = \left(r - 0.5\sigma^2\right) \Delta t + \ln(S_{t-1})$. Here, r is the risk-free rate, σ is the volatility, and S_{t-1} is the asset price at the previous step.

Once the multivariate state representing $\{S_1, S_2, \ldots, S_d\}$ is prepared, a quantum circuit computes the arithmetic average $\bar{S}$ by applying a weighted sum operator with equal weights $1/d$. The resulting register contains the average price:

$$|\bar{S}\rangle = |\frac{1}{d} \sum_{t=1}^{d} S_t\rangle \,.$$

Finally, the comparator and controlled-rotation circuits are used to encode the option payoff $\max(0, \bar{S} - K)$ into the payoff qubit. This process is analogous to that of single-asset options, with the difference that the asset price S_T at maturity is replaced by $\bar{S}$, the average price over the time period.

2. Barrier Options

Barrier options are unique in that their activation or deactivation depends on whether the underlying asset price crosses a pre-defined level, known as the barrier, during the option's life. These options can be categorized into two main types:

- **Knock-In Options:** These options only become active if the asset price crosses the barrier.
- **Knock-Out Options:** These options become void if the asset price crosses the barrier.

For a Knock-In option with a barrier level B, the payoff can be expressed as:

$$f(S) = \begin{cases} \max(0, S_T - K), & \text{if } \exists t \text{ such that } S_t \geq B, \\ 0, & \text{otherwise.} \end{cases}$$

Here, S_T is the asset price at maturity, K is the strike price, and S_t is the asset price at time t.

On a quantum computer, the price of the asset at each time step is represented using quantum registers, similar to the Asian option setup. The barrier condition introduces additional path dependence, which requires tracking whether the asset price has crossed B at any time step. This is achieved using:

- A register of d qubits, where each qubit indicates whether the barrier was crossed at a specific time step.
- An ancillary qubit that computes the logical OR of these barrier indicators, resulting in a single bit indicating whether the barrier was crossed at any point.

The logical OR operation is implemented using X (NOT) and Toffoli gates, ensuring that the barrier condition is properly encoded. For Knock-In options, the ancillary barrier qubit is used as a control to activate the payoff rotation. For Knock-Out options, an additional X gate is applied to invert the condition, deactivating the payoff if the barrier is crossed.

Finally, the comparator circuit is used to check if $S_T \geq K$, and controlled rotations encode the payoff into the payoff qubit:

$$\text{Payoff Qubit} = \begin{cases} \max(0, S_T - K), & \text{if the barrier condition is met,} \\ 0, & \text{otherwise.} \end{cases}$$

3. Asian Barrier Spreads

Asian barrier spreads combine features of Asian options and barrier options with a bull spread structure. The payoff depends on the average price of the asset over time (Asian component) but becomes zero if a barrier threshold B is breached at any point during the time horizon (Barrier component). Additionally, the payoff follows a piecewise linear function capped at a maximum value (Spread component).

The payoff function is defined as:

$$f_P(S_1, S_2) = \begin{cases} \min\left\{\max\left\{\frac{1}{2}(S_1 + S_2) - K_1, 0\right\}, K_2 - K_1\right\}, & \text{if } S_1, S_2 \leq B, \\ 0, & \text{otherwise,} \end{cases}$$

where S_1 and S_2 are the asset prices at two discrete times, K_1 and K_2 are the strike prices ($K_1 < K_2$), and B is the barrier threshold.

VII. RESULTS

All the code for the simulations and the generation of the plots presented in this section is provided in the Appendix. Most of the code is adapted and borrowed from the official documentation of *Qiskit Finance*.

A. European Call Option

In this section, we discuss the results for a European call option, which is a path-independent option where the payoff depends on whether the spot price of the underlying asset exceeds the strike price at maturity.

Figure 5 illustrates the probability distribution of the underlying asset price at maturity, loaded into a quantum register. The underlying asset price is modeled using the Black-Scholes-Merton framework, assuming a log-normal distribution for the asset price at maturity T. The quantum state prepared encodes this distribution over a three-qubit

register. Each computational basis state corresponds to a discretized asset price, and the probability amplitude represents the likelihood of that price.

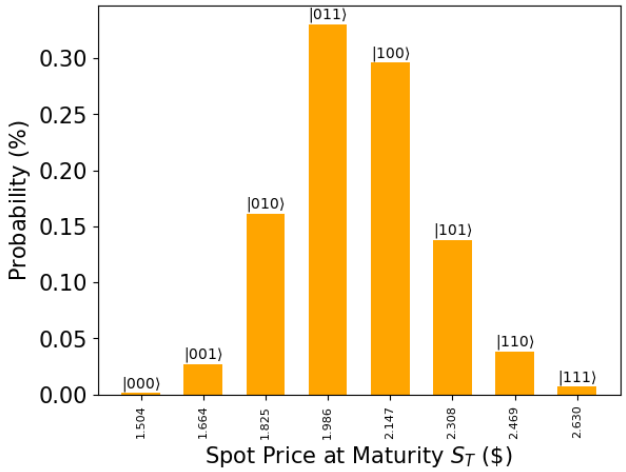


FIG. 5. Probability distribution of the underlying asset price at maturity, modeled using a log-normal distribution. The state is loaded into a three-qubit register using quantum gates, with each computational basis state corresponding to a discretized spot price, where $S_0 = 2$, $\sigma = 10\%$, $r = 4\%$, and $T = 300/365$.

Figure 6 shows the corresponding payoff function of the European call option. The payoff is zero when the asset price is below the strike price and increases linearly when the price exceeds the strike. This linear structure is mapped to the quantum circuit using controlled rotations, ensuring the payoff is correctly encoded in the quantum state.

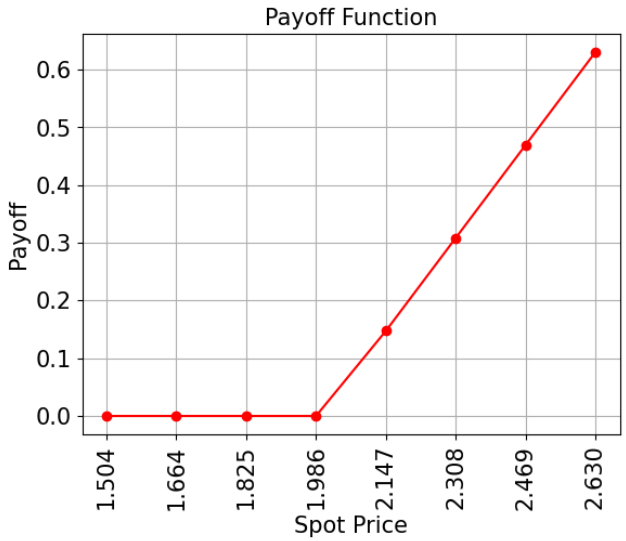


FIG. 6. Payoff function for the European call option. The payoff remains zero for prices below the strike and increases linearly for prices above the strike, as per the function $f(S_T) = \max(0, S_T - K)$.

Figure 7 illustrates the Monte Carlo simulation of possible stock price paths, modeled using a stochastic process for the underlying asset price. The plot shows only the first 10 simulated paths for clarity, capturing the random fluctuations in the asset price over the considered time steps. These simulations are fundamental for understanding the range of

potential asset prices at maturity, which directly influence the payoff of the option.

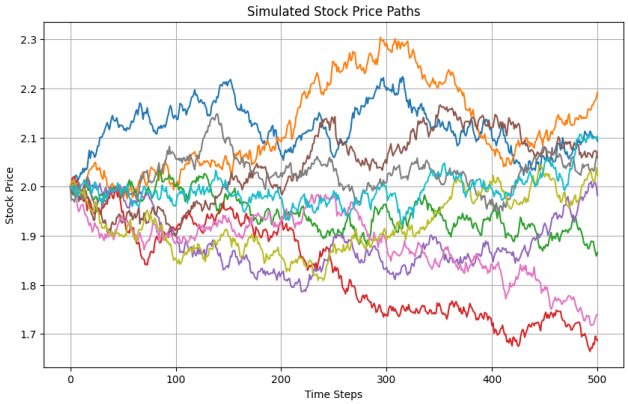


FIG. 7. Monte Carlo simulation of possible stock price paths. The plot contains only the first 10 simulations for clarity, showcasing the stochastic evolution of the asset price over time.

Figure 8 demonstrates the convergence of the option price estimate using Monte Carlo simulations. As the number of simulated paths increases, the estimated option price stabilizes, approaching the true value. This gradual convergence highlights the computational cost of classical Monte Carlo methods, where accuracy scales with $O(1/\sqrt{M})$, requiring a large number of samples for precise results.

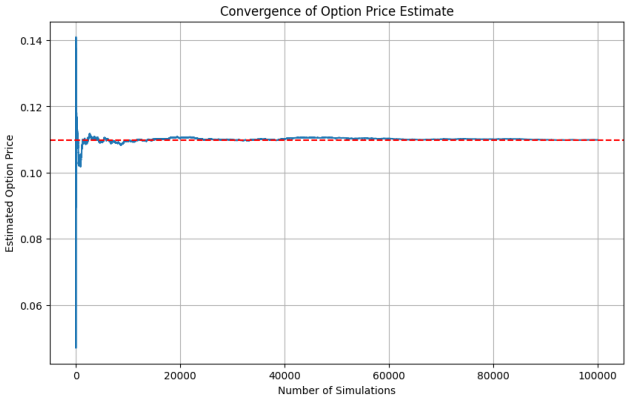


FIG. 8. Convergence of the estimated option price as a function of the number of Monte Carlo simulations. The red dashed line represents the true value of the option price, calculated analytically.

The comparison between Monte Carlo and quantum methods for loading probability distributions is illustrated in Figure 9. The blue curve represents the quantum-loaded probability distribution, while the black curve corresponds to the classical Monte Carlo method. Quantum amplitude estimation exhibits superior efficiency in generating these distributions, as it achieves quadratic speedup in convergence compared to Monte Carlo methods. However, the fidelity of the quantum state preparation for loading distributions depends on the number of qubits and the precision of the quantum circuits used.

Figure 10 shows the results from amplitude estimation applied to a European call option with a strike price $K = 2.0$ using $m = 3$ qubits. The red dashed line indicates the (undiscounted) analytical value from the Black-Scholes-Merton model. For $m = 3$, the results deviate significantly from the true value due to the low number of quantum samples ($M = 2^m$), as discussed in the original paper. This highlights the trade-off between

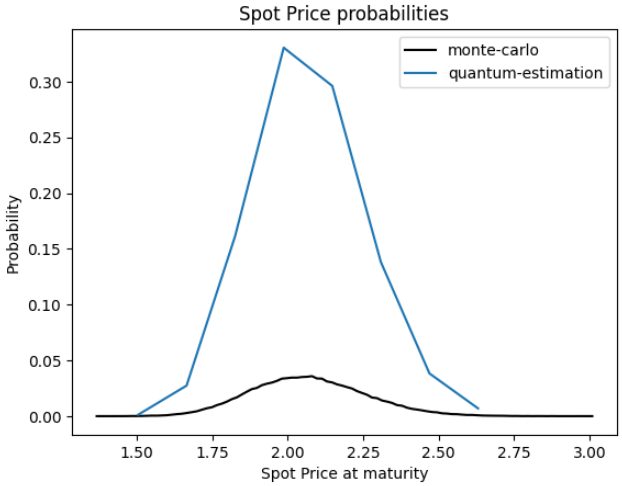


FIG. 9. Comparison between Monte Carlo and quantum amplitude estimation for loading probability distributions at maturity. The quantum method demonstrates better efficiency, with faster convergence to the desired distribution.

quantum resources and accuracy in practical implementations of amplitude estimation.

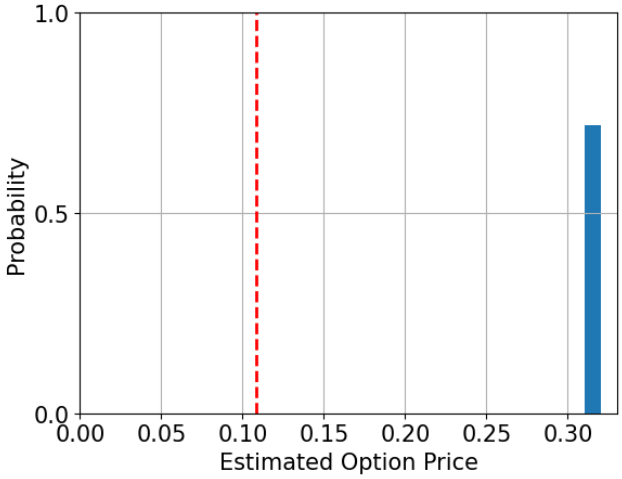


FIG. 10. Results from amplitude estimation on a European call option with a strike price $K = 2.0$ using $m = 3$ qubits. The red dashed line indicates the analytical value from the Black-Scholes-Merton model. Results show that low qubit counts lead to higher error in the option price estimation.

Figure 11 shows how amplitude estimation achieves improved accuracy as the qubit count m increases, converging to the exact option price. This is almost similar in behavior to the plot produced in the original paper Figure 12, where higher qubit registers provide increasingly refined estimates.

The quantum approach to pricing European call options demonstrates significant advantages in computational efficiency and convergence rates. While lower qubit numbers produce suboptimal results, increasing the qubit register size consistently narrows the gap between quantum estimates and analytical values. These results validate the theoretical speedup and emphasize the scalability of quantum methods for complex financial instruments.

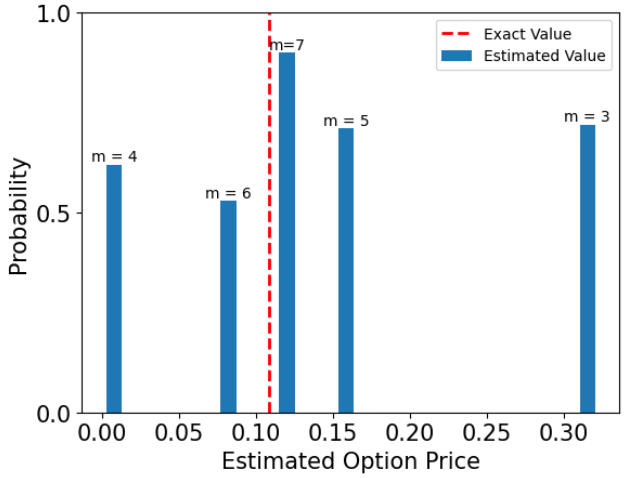


FIG. 11. Amplitude estimation results for a European call option with strike $K = 2.0$ using $m = \{3, 4, 5, 6, 7\}$ qubits. The red dashed line shows the exact analytical value from the Black-Scholes-Merton model.

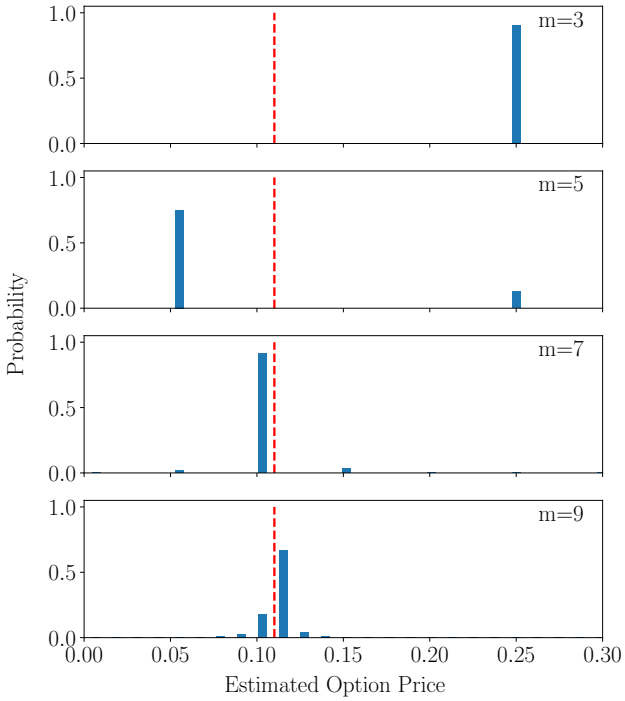


FIG. 12. Comparison of amplitude estimation results for $m = \{3, 5, 7, 9\}$ qubits. The red dashed line represents the exact analytical value from the Black-Scholes-Merton model. This plot is in the original paper.

B. Asian Barrier Spreads

Asian barrier spreads combine features of Asian options, barrier options, and spread options. This unique hybrid option involves averaging, barriers, and a piecewise linear payoff structure. The following results demonstrate how these aspects are captured in the

quantum framework.

Figure 5 shows the probability distribution of two spot prices, S_1 and S_2, modeled using a log-normal distribution. The plot demonstrates the likelihood of different combinations of S_1 and S_2, providing insight into how the uncertainty model is represented on the quantum register. The peaks highlight the regions with the highest probability density, crucial for accurate option pricing.

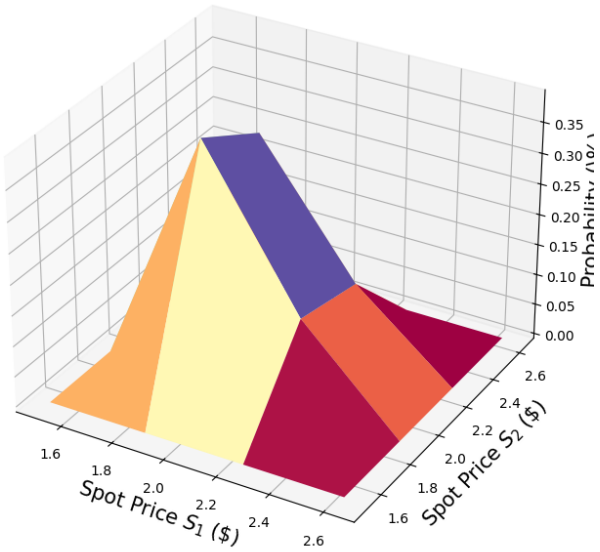


FIG. 13. Probability Distribution Function of the Uncertainty Model for spot prices $S_1 = 2.0$ and $S_2 = 2.0$. The log-normal distribution reflects realistic market behavior, ensuring reliable pricing outcomes.

Figure 14 depicts the exact payoff function for the Asian Barrier Spread when $S_1 = S_2$. The piecewise linear payoff begins at zero, increases linearly within the spread range $K_1 < \bar{S} < K_2$, and then levels off. This illustrates the combined effect of averaging and spread constraints.

Figure 15 expands the analysis to consider the barrier condition. It visualizes the payoff function with respect to the two time steps $\Delta t = T/2$ and $\Delta t = T$. The gradient in the plot reflects the changing payoff as a function of S_1 and S_2, with the barrier condition set at $B = 6.0$. The plot effectively captures the path-dependent nature of the option.

Figure 16 presents the quantum estimation results for the Asian Barrier Spread option price. For strikes $K_1 = 4.0$ and $K_2 = 5.0$, the estimated value (blue bar) closely matches the exact analytical value (red dashed line). This result demonstrates the accuracy of the quantum implementation in pricing complex options.

The results presented for Asian Barrier Spreads highlight the power of quantum algorithms in pricing complex financial derivatives. The quantum framework effectively captures path dependency, averaging, and barrier constraints, as evidenced by the precise alignment between the estimated and exact values. The visualization of the probability distribution, payoff function, and estimation results further underscores the robustness of this methodology in solving computationally intensive problems in finance.

VIII. QUANTUM HARDWARE RESULTS AND ERROR MITIGATION

In this section, I analyze the implementation of amplitude estimation (AE) circuits for European call options on noisy quantum hardware as presented in the original paper, high-

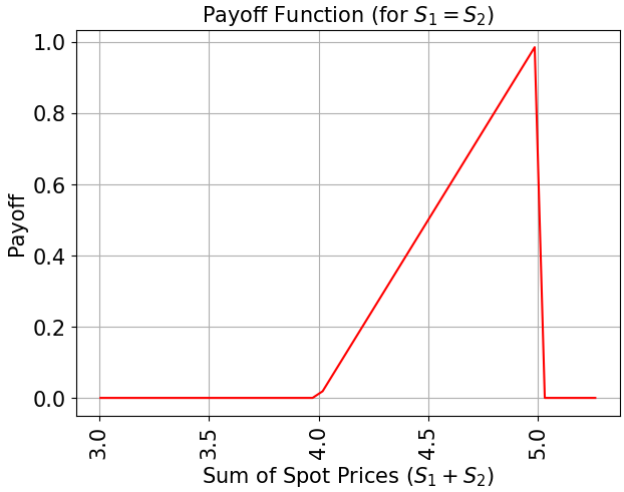


FIG. 14. Exact payoff function for the Asian Barrier Spread when $S_1 = S_2$. The function demonstrates the interplay of the spread and barrier constraints.

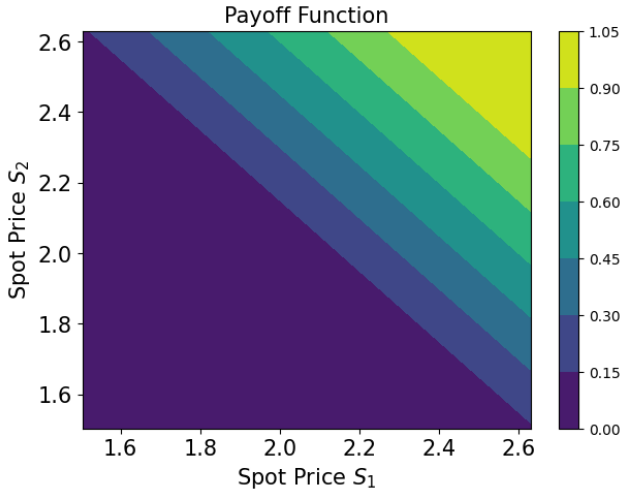


FIG. 15. Payoff function across two time steps ($\Delta t = T/2, \Delta t = T$), incorporating a barrier at $B = 6.0$. The plot highlights the barrier's influence on the payoff structure.

lighting resource requirements, feasibility, and error mitigation strategies. Our focus is on AE without quantum phase estimation, which significantly reduces the circuit depth and complexity, making it more suitable for near-term quantum devices. Note that, I am just understanding and presenting the results of the original paper in this section and not claiming them to be of my own.

A. Resource Requirements for AE Circuits

Amplitude estimation circuits demand substantial resources due to multi-qubit gates and sampling requirements. Table II provides a breakdown of single-qubit gates, CNOT gates, Toffoli gates, and overall circuit depth for AE circuits as a function of the number of sampling qubits m.

The exponential scaling of gate counts with m illustrates the impracticality of standard

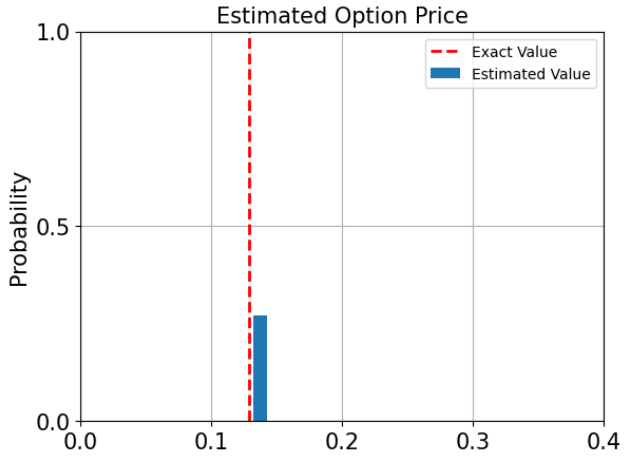


FIG. 16. Results from Amplitude Estimation on an Asian Barrier Spread option with strikes $K_1 = 4.0$ and $K_2 = 5.0$. The red dashed line denotes the exact analytical value, while the blue bar represents the quantum-derived estimate.

# Sampling Qubits (m)	Single-Qubit Gates	CNOT Gates	Toffoli Gates	Circuit Depth
$m = 3$	2,091	2,056	90	3,927
$m = 5$	12,768	9,078	378	17,332
$m = 7$	52,275	37,132	1,530	70,916
$m = 9$	210,144	149,290	6,138	285,204

TABLE II. Resource requirements for amplitude estimation circuits as a function of sampling qubits m, assuming all-to-all qubit connectivity.

AE circuits for near-term devices. These demands highlight the need for efficient implementations and error mitigation strategies.

B. Error Mitigation Techniques

Noisy quantum hardware introduces errors that significantly impact computation. To address these, we implement two primary error mitigation techniques: **readout error mitigation** and **Richardson extrapolation**.

1. Readout Error Mitigation

Readout errors occur during the measurement process, leading to incorrect state assignments. To correct these errors, we construct a readout error matrix $\mathcal{R}$, where each element $\mathcal{R}_{ij}$ represents the probability of measuring state j when state i is prepared (Dewes *et al.*, 2012). The corrected measurement probabilities are obtained by solving the system:

$$\mathbf{P}_{\text{measured}} = \mathcal{R} \cdot \mathbf{P}_{\text{true}}.$$

Inverting $\mathcal{R}$ allows us to estimate the true probabilities $\mathbf{P}_{\text{true}}$:

$$\mathbf{P}_{\text{true}} = \mathcal{R}^{-1} \cdot \mathbf{P}_{\text{measured}}.$$

Using this method, the error in estimating P_1 for the $\mathcal{A}|0\rangle_3$ circuit was reduced from approximately 6% to 4%.

2. Richardson Extrapolation for Circuit Noise

Richardson extrapolation is a technique to estimate the zero-noise limit of a computation by running the circuit at multiple noise levels. The noise is artificially amplified by inserting redundant gates, such as additional CNOT gates, into the circuit.

The measured observable $E(\lambda)$, where λ represents the noise strength, can be expanded as:

$$E(\lambda) = E(0) + c_1\lambda + c_2\lambda^2 + \dots,$$

where $E(0)$ is the zero-noise result we aim to estimate. By running the circuit at scaled noise levels $\lambda, k\lambda, \dots$, and obtaining $E(\lambda), E(k\lambda), \dots$, we can eliminate higher-order terms through extrapolation. For example, with two noise levels λ and $k\lambda$:

$$E(0) \approx \frac{kE(\lambda) - E(k\lambda)}{k-1}.$$

To improve accuracy, higher-order Richardson extrapolation can be used, where the extrapolated result is obtained as:

$$E(0) \approx E(\lambda) - \frac{E(k\lambda) - E(\lambda)}{k-1} + \mathcal{O}(\lambda^3).$$

The error scales as $\mathcal{O}(\lambda^3)$, offering a more accurate estimation of $E(0)$ (Temme, Bravyi, and Gambetta, 2017).

In the experiments, noise was amplified by replacing each CNOT gate with one, three, or five consecutive CNOT gates. The circuit was executed with these gate counts, corresponding to noise levels $\lambda, 3\lambda$, and 5λ. Using second-order Richardson extrapolation, the zero-noise result was estimated. Figure 17(b) illustrates the behavior of the noisy results and the extrapolated zero-noise limit.

This approach successfully reduced the average error in estimated option prices from 62% to 21%, demonstrating its effectiveness.

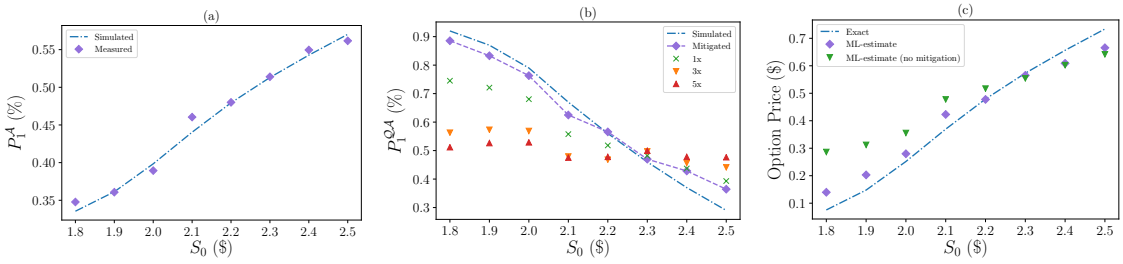


FIG. 17. Error Mitigated Results for: (a) Measured probabilities for $\mathcal{A}|0\rangle_3$, (b) Measured probabilities for $\mathcal{QA}|0\rangle_3$ with varying noise levels, and (c) Option price estimated after Richardson extrapolation, compared to classical results.

C. Circuit Optimization and Results

The $\mathcal{A}$ operator for the European call option was constructed using a log-normal distribution on two qubits, encoded with a single CNOT gate and four single-qubit rotations. The payoff function was applied using a uniformly controlled Y-rotation, efficiently implemented with four CNOT gates and four single-qubit rotations. The total circuit for $\mathcal{QA}|0\rangle_3$ consisted of 18 CNOT gates and 33 single-qubit gates after optimization.

Experiments were conducted on the *IBM Q Tokyo* 20-qubit device. The results for different initial spot prices $S_0 \in [1.8, 2.5]$ showed that:

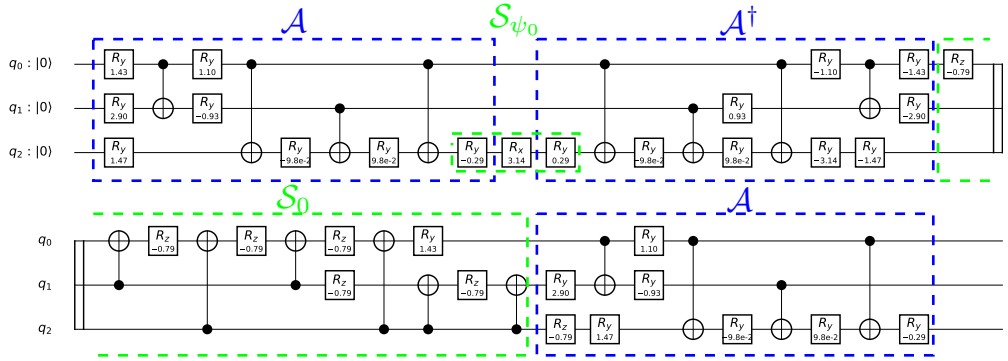


FIG. 18. The optimized quantum circuit for $\mathcal{QA}\,|0\rangle_3$, employed in the experiments on real quantum hardware, consists of 18 CNOT gates and 33 single-qubit gates. The initial spot price is set to 2 for these experiments. Dashed boxes in the circuit diagram indicate the components corresponding to $\mathcal{A}$, $\mathcal{A}^\dagger$, $\mathcal{S}_{\psi_0}$, and $\mathcal{S}_0$. Due to circuit optimization techniques, some of these components overlap slightly.

- Without error mitigation, hardware noise significantly affected the accuracy of option price estimates.
- Readout error mitigation reduced measurement errors but did not address gate-level noise.
- Richardson extrapolation successfully recovered accurate results, converging to classical simulations.

IX. CONCLUSION

In this report, I explored a quantum methodology for pricing financial options and portfolios, presenting an implementation framework and analyzing quantum circuits tailored for these tasks. By leveraging amplitude estimation (AE), a quantum algorithm that provides quadratic speed-up over traditional Monte Carlo simulations. The authors of the original paper demonstrated this quantum advantage for a basket option with three equally weighted assets. Figure 19 illustrates the comparison of estimation errors between AE and Monte Carlo simulations at an 81% confidence interval. While AE achieves $\mathcal{O}(M^{-1})$ convergence, Monte Carlo exhibits $\mathcal{O}(M^{-1/2})$ scaling, consistent with its classical limitations. This figure highlights the potential of quantum algorithms like AE to outperform traditional methods, particularly for high-precision calculations in the financial industry. I demonstrated how quantum techniques could handle complex features of exotic options, such as path-dependency, barriers, and averages. While the implementation was carried out in a simulated environment, I discussed the original authors' findings on running these circuits on actual noisy quantum hardware, shedding light on the practical challenges and solutions.

The results highlight the potential of quantum computing to revolutionize financial modeling. In particular, the use of advanced AE-based algorithms, such as the one developed by Suzuki *et al.*, achieves significant resource optimization by reducing gate counts while retaining the quadratic speed-up. Additionally, I detailed error mitigation techniques like Richardson extrapolation, which helped reduce computational errors on noisy hardware, albeit within the limitations of current quantum devices.

Despite these advancements, significant hurdles remain. Current quantum hardware lacks the capacity to execute the deep and complex circuits required for pricing typical financial portfolios. Furthermore, achieving practical quantum advantage in this domain will likely

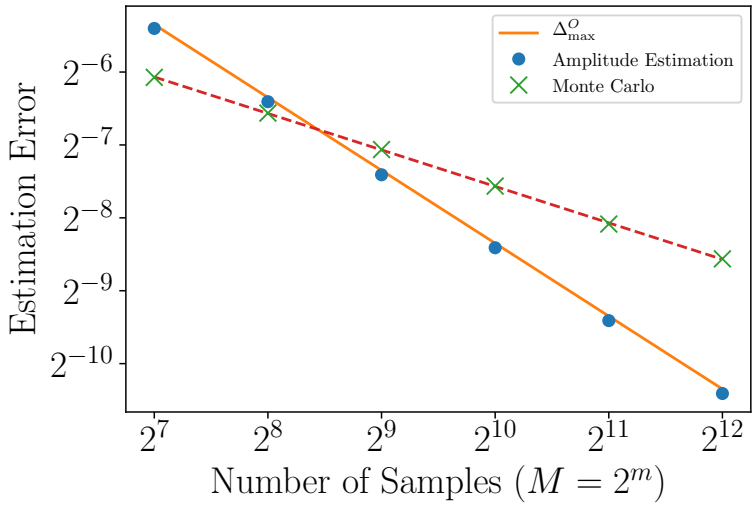


FIG. 19. Comparison of estimation errors between Amplitude Estimation and Monte Carlo at an 81% confidence interval for a basket option with three identical, equally weighted assets. Parameters include a strike price of $K = 2.0$ and asset correlations $\rho_{12} = \rho_{13} = \rho_{23} = 0.8$. Amplitude Estimation shows $\mathcal{O}(M^{-1})$ convergence, while Monte Carlo errors, calculated over 10,000 simulations for each sample 2^m, exhibit the expected $\mathcal{O}(M^{-1/2})$ scaling, with a linear fit (red dashed line) Stamatopoulos *et al.* (2020).

require universal, fault-tolerant quantum computers. Future efforts could focus on further optimizing quantum algorithms to reduce qubit requirements and circuit depth, enabling the use of quantum computing for large-scale financial applications. As quantum hardware and algorithms continue to evolve, the techniques discussed here could pave the way for the efficient pricing of financial derivatives on quantum computers.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Prof. Colin Benjamin for his insightful guidance and encouragement throughout this computational laboratory course. His constructive comments and insightful questions during the presentation have greatly contributed to a deeper understanding of the concepts explored in this project.

I am also deeply thankful to the teaching assistants, Rajmohan Dora and Dinesh Kumar Panda, for their constant support and for addressing my queries with patience and clarity.

Lastly, I would like to extend my heartfelt thanks to my colleagues for their collaborative spirit and constructive discussions.

IMPLEMENTATION USING QISKIT

Appendix A: European Call Option Pricing

1. Loading the Probability Distribution

```
%pip install qiskit-finance qiskit-algorithms qiskit-aer pylatexenc
import matplotlib.pyplot as plt

%matplotlib inline
```

```
import numpy as np

from qiskit import QuantumCircuit, transpile
from qiskit_algorithms import IterativeAmplitudeEstimation, EstimationProblem
                                    , AmplitudeEstimation
from qiskit.circuit.library import LinearAmplitudeFunction
from qiskit_finance.circuit.library import LogNormalDistribution
from qiskit_finance.applications.estimation import EuropeanCallPricing

from qiskit.visualization import plot_histogram
from qiskit.primitives import Sampler as RefSampler

from qiskit_aer import AerSimulator

# number of qubits to represent the uncertainty
num_uncertainty_qubits = 3

# parameters for considered random distribution
S = 2  # initial spot price
vol = 0.1  # volatility of 10%
r = 0.04  # annual interest rate of 4%
T = 300 / 365  # 300 days to maturity

# resulting parameters for log-normal distribution
mu = (r - 0.5 * vol**2) * T + np.log(S)
sigma = vol * np.sqrt(T)
mean = np.exp(mu + sigma**2 / 2)
variance = (np.exp(sigma**2) - 1) * np.exp(2 * mu + sigma**2)
stddev = np.sqrt(variance)

# lowest and highest value considered for the spot price; in between, an
                                    equidistant discretization is
                                    considered.
low = np.maximum(0, mean - 3 * stddev)
high = mean + 3 * stddev

# construct A operator for QAE for the payoff function by
# composing the uncertainty model and the objective
uncertainty_model = LogNormalDistribution(
    num_uncertainty_qubits, mu=mu, sigma=sigma**2, bounds=(low, high)
)

# plot probability distribution
x = uncertainty_model.values
y = uncertainty_model.probabilities

labels = [r"$|000\rangle$", r"$|001\rangle$", r"$|010\rangle$", r"$|011\
                                    rangle$",
          r"$|100\rangle$", r"$|101\rangle$", r"$|110\rangle$", r"$|111\
                                        rangle$"]

bars = plt.bar(x, y, width=0.1, color="orange")
plt.xticks(x, size=8, rotation=90)
plt.yticks(size=15)
plt.xlabel("Spot Price at Maturity $S_T$ (\$)", size=15)
plt.ylabel("Probability ($\%$)", size=15)

# for i, bar in enumerate(bars):
#     plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), labels[i
                                    ],
#              ha='center', va='bottom', fontsize=10)

plt.show()
```

2. Computing the Payoff Function

```
# set the strike price (should be within the low and the high value of the
                                        uncertainty)
strike_price = 2

# set the approximation scaling for the payoff function
c_approx = 0.25

# setup piecewise linear objective fcuntion
breakpoints = [low, strike_price]
slopes = [0, 1]
offsets = [0, 0]
f_min = 0
f_max = high - strike_price
european_call_objective = LinearAmplitudeFunction(
    num_uncertainty_qubits,
    slopes,
    offsets,
    domain=(low, high),
    image=(f_min, f_max),
    breakpoints=breakpoints,
    rescaling_factor=c_approx,
)

# construct A operator for QAE for the payoff function by
# composing the uncertainty model and the objective
num_qubits = european_call_objective.num_qubits
european_call = QuantumCircuit(num_qubits)
european_call.append(uncertainty_model, range(num_uncertainty_qubits))
european_call.append(european_call_objective, range(num_qubits))

# plot exact payoff function (evaluated on the grid of the uncertainty model)
x = uncertainty_model.values
y = np.maximum(0, x - strike_price)
plt.plot(x, y, "ro-")
plt.grid()
plt.title("Payoff Function", size=15)
plt.xlabel("Spot Price", size=15)
plt.ylabel("Payoff", size=15)
plt.xticks(x, size=15, rotation=90)
plt.yticks(size=15)
plt.show()
```

3. Evaluations for Final Result Plot

```
# evaluate exact expected value (normalized to the [0, 1] interval)
exact_value = np.dot(uncertainty_model.probabilities, y)
exact_delta = sum(uncertainty_model.probabilities[x >= strike_price])
print("exact expected value:\t%.4f" % exact_value)

european_call_pricing = EuropeanCallPricing(
    num_state_qubits=num_uncertainty_qubits,
    strike_price=strike_price,
    rescaling_factor=c_approx,
    bounds=(low, high),
    uncertainty_model=uncertainty_model,
)

# set target precision and confidence level
epsilon = 0.01
alpha = 0.05
```

```
problem = european_call_pricing.to_estimation_problem()
# construct amplitude estimation
ae = AmplitudeEstimation(
    # epsilon_target=epsilon,
    4,
    sampler=RefSampler(options={"seed": 75, "shots": 100}),
)

result = ae.estimate(problem)

conf_int = np.array(result.confidence_interval_processed)
estimated_value = european_call_pricing.interpret(result)
print("Exact value:        \t%.4f" % exact_value)
print("Estimated value:    \t%.4f" % estimated_value)
print("Confidence interval:\t[%.4f, %.4f]" % tuple(conf_int))

ae_iter = IterativeAmplitudeEstimation(
    epsilon_target=epsilon, alpha=alpha, sampler=RefSampler(options={"shots":
                                          100, "seed": 75})
)
ae_iter_result = ae_iter.estimate(problem)
expected_iter = european_call_pricing.interpret(ae_iter_result)
conf_int = np.array(ae_iter_result.confidence_interval_processed)
print("Exact value:        \t%.4f" % exact_value)
print("Estimated value:    \t%.4f" % expected_iter)
print("Confidence interval:\t[%.4f, %.4f]" % tuple(conf_int))

ae4 = AmplitudeEstimation(
    # epsilon_target=epsilon,
    4,
    sampler=RefSampler(options={"seed": 75, "shots": 100}),
)

result4 = ae4.estimate(problem)

ae5 = AmplitudeEstimation(
    # epsilon_target=epsilon,
    5,
    sampler=RefSampler(options={"seed": 75, "shots": 100}),
)

result5 = ae5.estimate(problem)

ae6 = AmplitudeEstimation(
    # epsilon_target=epsilon,
    6,
    sampler=RefSampler(options={"seed": 75, "shots": 100}),
)

result6 = ae6.estimate(problem)

ae7 = AmplitudeEstimation(
    # epsilon_target=epsilon,
    7,
    sampler=RefSampler(options={"seed": 75, "shots": 100}),
)

result7 = ae7.estimate(problem)

results = [result, result4, result5, result6, result7]
bar_results = []
probs = []
for i in results:
    bar_results.append(european_call_pricing.interpret(i))
    probs.append(i.max_probability)
```

4. Possible Price Paths, Monte Carlo Convergence and Comparison of Quantum Estimation and Monte Carlo

```
N = 10**5                          # no. of simulations in monte carlo approach
M = 500                            # steps per approach in monte carlo approach

# Time step
dt = T / M

# Simulate the price paths
path = np.zeros((N, M + 1))
path[:, 0] = S
for t in range(1, M + 1):
    z = np.random.standard_normal(N)   # Generate random variables
    path[:, t] = path[:, t-1] * np.exp((r - 0.5 * sigma**2) * dt + vol * np.
                                       sqrt(dt) * z)

# Calculate the payoff at maturity
payoff = np.maximum(path[:, -1] - strike_price, 0)

# Discount the payoff to present value
discounted_payoff = np.exp(-r * T) * payoff

# Calculate the option price for increasing number of simulations to show
                                    convergence
prices = np.zeros(N)
for i in range(1, N + 1):
    prices[i-1] = np.mean(discounted_payoff[:i])

option_price = prices[-1]
print(f"European Call Option Price: {option_price:.2f}")


# Plotting some of the simulated price paths
plt.figure(figsize=(10, 6))
for i in range(min(N, 10)):          # Plotting only the first 10 simulations
                                       for clarity
    plt.plot(path[i, :], lw=1.5)
plt.title('Simulated Stock Price Paths')
plt.xlabel('Time Steps')
plt.ylabel('Stock Price')
plt.grid(True)
plt.show()


# Plotting the convergence of the option price
plt.figure(figsize=(10, 6))
plt.plot(prices, lw=1.5)
plt.title('Convergence of Option Price Estimate')
plt.xlabel('Number of Simulations')
plt.ylabel('Estimated Option Price')
plt.grid(True)
plt.axhline(y=option_price, color='r', linestyle='--')
plt.show()

plt.figure()

# Plot the probabilities according to monte carlo
y, binEdges = np.histogram(path[:, -1], bins=100)
bincenters = 0.5 * (binEdges[1:] + binEdges[:-1])
plt.plot(bincenters, y/N, '-', c='black', label="monte-carlo")


# plot the probabilities according to quantum estimation
x = uncertainty_model.values
y = uncertainty_model.probabilities
```

```
plt.plot(x, y, label="quantum-estimation")


# Set graph labels
plt.title("Spot Price probabilities")
plt.xlabel("Spot Price at maturity")
plt.ylabel("Probability")
plt.legend()


plt.show()
```

5. Final Plot

```
bars = plt.bar(bar_results, probs, width=0.01, label='Estimated Value')
plt.plot([exact_value, exact_value], [0,1], 'r--', linewidth=2, label='Exact
                                        Value')
plt.xticks([0.0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30], size=15)
plt.yticks([0.0, 0.5, 1], size=15)
plt.xlabel('Estimated Option Price', size=15)
plt.ylabel('Probability', size=15)
plt.ylim((0,1))

labels = ["m = 3", "m = 4", "m = 5", "m = 6", "m=7"]

plt.legend()

for i, bar in enumerate(bars):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height(), labels[i],
             ha='center', va='bottom', fontsize=10)
plt.show()
```

Appendix B: Asian Barrier Spread Option Pricing

1. Loading the Probability Distribution

```
import matplotlib.pyplot as plt
from scipy.interpolate import griddata

%matplotlib inline
import numpy as np

from qiskit import QuantumRegister, QuantumCircuit, AncillaRegister,
                                        transpile
from qiskit.circuit.library import IntegerComparator, WeightedAdder,
                                        LinearAmplitudeFunction
from qiskit_algorithms import IterativeAmplitudeEstimation, EstimationProblem
from qiskit.primitives import Sampler as RefSampler
from qiskit_finance.circuit.library import LogNormalDistribution

# number of qubits per dimension to represent the uncertainty
num_uncertainty_qubits = 2

# parameters for considered random distribution
S = 2.0  # initial spot price
vol = 0.1  # volatility of 10%
r = 0.04  # annual interest rate of 4%
T = 300 / 365  # 300 days to maturity
```

```
dt = T / 2

# resulting parameters for log-normal distribution
mu = (r - 0.5 * vol**2) * T + np.log(S)
sigma = vol * np.sqrt(T)
mean = np.exp(mu + sigma**2 / 2)
variance = (np.exp(sigma**2) - 1) * np.exp(2 * mu + sigma**2)
stddev = np.sqrt(variance)

# lowest and highest value considered for the spot price; in between, an
                                        equidistant discretization is
                                        considered.
low = np.maximum(0, mean - 3 * stddev)
high = mean + 3 * stddev

# map to higher dimensional distribution
# for simplicity assuming dimensions are independent and identically
                                        distributed)
dimension = 2
num_qubits = [num_uncertainty_qubits] * dimension
low = low * np.ones(dimension)
high = high * np.ones(dimension)
mu = [(r - 0.5 * vol**2) * dt + np.log(S), (r - 0.5 * vol**2) * (2 * dt) + np
                                        .log(S)]
cov = dt * sigma**2 * np.eye(dimension)

# construct circuit
u = LogNormalDistribution(num_qubits=num_qubits, mu=mu, sigma=cov, bounds=(
                                        list(zip(low, high))))

# plot PDF of uncertainty model
x = [v[0] for v in u.values]
y = [v[1] for v in u.values]
z = u.probabilities
# z = map(float, z)
# z = list(map(float, z))
resolution = np.array([2**n for n in num_qubits]) * 1j
grid_x, grid_y = np.mgrid[min(x) : max(x) : resolution[0], min(y) : max(y) :
                                        resolution[1]]
grid_z = griddata((x, y), z, (grid_x, grid_y))
plt.figure(figsize=(10, 8))
ax = plt.axes(projection="3d")
ax.plot_surface(grid_x, grid_y, grid_z, cmap=plt.cm.Spectral)
ax.set_xlabel("Spot Price $S_1$ (\$)", size=15)
ax.set_ylabel("Spot Price $S_2$ (\$)", size=15)
ax.set_zlabel("Probability (\%)", size=15)
plt.show()
```

2. Computing the Payoff Function

```
# determine number of qubits required to represent total loss
weights = []
for n in num_qubits:
    for i in range(n):
        weights += [2**i]

# create aggregation circuit
agg = WeightedAdder(sum(num_qubits), weights)
n_s = agg.num_sum_qubits
n_aux = agg.num_qubits - n_s - agg.num_state_qubits  # number of additional
                                        qubits

# set the strike price (should be within the low and the high value of the
                                        uncertainty)
```

```
strike_price_1 = 4
strike_price_2 = 5

# set the barrier threshold
barrier = 6

# map strike prices and barrier threshold from [low, high] to {0, ..., 2^n-1}
max_value = 2**n_s - 1
low_ = low[0]
high_ = high[0]

mapped_strike_price_1 = (
    (strike_price_1 - dimension * low_) / (high_ - low_) * (2**
                                            num_uncertainty_qubits - 1)
)
mapped_strike_price_2 = (
    (strike_price_2 - dimension * low_) / (high_ - low_) * (2**
                                            num_uncertainty_qubits - 1)
)
mapped_barrier = (barrier - low) / (high - low) * (2**num_uncertainty_qubits
                                        - 1)

# condition and condition result
conditions = []
barrier_thresholds = [2] * dimension
n_aux_conditions = 0
for i in range(dimension):
    # target dimension of random distribution and corresponding condition (
                                            which is required to be True)
    comparator = IntegerComparator(num_qubits[i], mapped_barrier[i] + 1, geq=
                                            False)
    n_aux_conditions = max(n_aux_conditions, comparator.num_ancillas)
    conditions += [comparator]

# set the approximation scaling for the payoff function
c_approx = 0.25

# setup piecewise linear objective fcuntion
breakpoints = [0, mapped_strike_price_1, mapped_strike_price_2]
slopes = [0, 1, 0]
offsets = [0, 0, mapped_strike_price_2 - mapped_strike_price_1]
f_min = 0
f_max = mapped_strike_price_2 - mapped_strike_price_1
objective = LinearAmplitudeFunction(
    n_s,
    slopes,
    offsets,
    domain=(0, max_value),
    image=(f_min, f_max),
    rescaling_factor=c_approx,
    breakpoints=breakpoints,
)

# define overall multivariate problem
qr_state = QuantumRegister(u.num_qubits, "state")  # to load the probability
                                        distribution
qr_obj = QuantumRegister(1, "obj")  # to encode the function values
ar_sum = AncillaRegister(n_s, "sum")  # number of qubits used to encode the
                                        sum
ar_cond = AncillaRegister(len(conditions) + 1, "conditions")
ar = AncillaRegister(
    max(n_aux, n_aux_conditions, objective.num_ancillas), "work"
)  # additional qubits

objective_index = u.num_qubits

# define the circuit
```

```
asian_barrier_spread = QuantumCircuit(qr_state, qr_obj, ar_cond, ar_sum, ar)

# load the probability distribution
asian_barrier_spread.append(u, qr_state)

# apply the conditions
for i, cond in enumerate(conditions):
    state_qubits = qr_state[(num_uncertainty_qubits * i) : (
                                            num_uncertainty_qubits * (i + 1))]
    asian_barrier_spread.append(cond, state_qubits + [ar_cond[i]] + ar[: cond
                                            .num_ancillas])

# aggregate the conditions on a single qubit
asian_barrier_spread.mcx(ar_cond[:-1], ar_cond[-1])

# apply the aggregation function controlled on the condition
asian_barrier_spread.append(agg.control(), [ar_cond[-1]] + qr_state[:] +
                                        ar_sum[:] + ar[:n_aux])

# apply the payoff function
asian_barrier_spread.append(objective, ar_sum[:] + qr_obj[:] + ar[: objective
                                        .num_ancillas])

# uncompute the aggregation
asian_barrier_spread.append(
    agg.inverse().control(), [ar_cond[-1]] + qr_state[:] + ar_sum[:] + ar[:
                                            n_aux]
)

# uncompute the conditions
asian_barrier_spread.mcx(ar_cond[:-1], ar_cond[-1])

for j, cond in enumerate(reversed(conditions)):
    i = len(conditions) - j - 1
    state_qubits = qr_state[(num_uncertainty_qubits * i) : (
                                            num_uncertainty_qubits * (i + 1))]
    asian_barrier_spread.append(
        cond.inverse(), state_qubits + [ar_cond[i]] + ar[: cond.num_ancillas]
    )

print(asian_barrier_spread.draw())
print("objective qubit index", objective_index)

# plot exact payoff function
plt.figure(figsize=(7, 5))
x = np.linspace(sum(low), sum(high))
y = (x <= 5) * np.minimum(np.maximum(0, x - strike_price_1), strike_price_2 -
                                         strike_price_1)
plt.plot(x, y, "r-")
plt.grid()
plt.title("Payoff Function (for $S_1 = S_2$)", size=15)
plt.xlabel("Sum of Spot Prices ($S_1 + S_2)$", size=15)
plt.ylabel("Payoff", size=15)
plt.xticks(size=15, rotation=90)
plt.yticks(size=15)
plt.show()

# plot contour of payoff function with respect to both time steps, including
                                        barrier
plt.figure(figsize=(7, 5))
z = np.zeros((100, 100))
x = np.linspace(low[0], high[0], 100)
y = np.linspace(low[1], high[1], 100)
for i, x_ in enumerate(x):
    for j, y_ in enumerate(y):
        z[i, j] = np.minimum(
```

```
            np.maximum(0, x_ + y_ - strike_price_1), strike_price_2 -
                                                    strike_price_1
        )
        if x_ > barrier or y_ > barrier:
            z[i, j] = 0

plt.title("Payoff Function", size=15)
plt.contourf(x, y, z)
plt.colorbar()
plt.xlabel("Spot Price $S_1$", size=15)
plt.ylabel("Spot Price $S_2$", size=15)
plt.xticks(size=15)
plt.yticks(size=15)
plt.show()
```

3. Evaluation for Final Result Plot

```
num_state_qubits = asian_barrier_spread.num_qubits - asian_barrier_spread.
                                        num_ancillas
print("state qubits: ", num_state_qubits)
transpiled = transpile(asian_barrier_spread, basis_gates=["u", "cx"])
print("circuit width:", transpiled.width())
print("circuit depth:", transpiled.depth())

asian_barrier_spread_measure = asian_barrier_spread.measure_all(inplace=False
                                        )
sampler = RefSampler()
job = sampler.run(asian_barrier_spread_measure)

# evaluate the result
value = 0
probabilities = job.result().quasi_dists[0].binary_probabilities()
for i, prob in probabilities.items():
    if prob > 1e-4 and i[-num_state_qubits:][0] == "1":
        value += prob


# map value to original range
mapped_value = objective.post_processing(value) / (2**num_uncertainty_qubits
                                        - 1) * (high_ - low_)
print("Exact Operator Value:  %.4f" % value)
print("Mapped Operator value: %.4f" % mapped_value)
print("Exact Expected Payoff: %.4f" % exact_value)
print(job.result().quasi_dists[0].binary_probabilities())

# set target precision and confidence level
epsilon = 0.01
alpha = 0.05

problem = EstimationProblem(
    state_preparation=asian_barrier_spread,
    objective_qubits=[objective_index],
    post_processing=objective.post_processing,
)
# construct amplitude estimation
ae = IterativeAmplitudeEstimation(
    epsilon, alpha=alpha, sampler=RefSampler(options={"shots": 100, "seed":
                                        75})
)

result = ae.estimate(problem)

conf_int = (
    np.array(result.confidence_interval_processed)
```

```
    / (2**num_uncertainty_qubits - 1)
    * (high_ - low_)
)
print("Exact value:    \t%.4f" % exact_value)
print(
    "Estimated value:\t%.4f"
    % (result.estimation_processed / (2**num_uncertainty_qubits - 1) * (high_
                                            - low_))
)
print("Confidence interval: \t[%.4f, %.4f]" % tuple(conf_int))

# plot estimated values for option price (after re-scaling and reversing the
                                        c_approx-transformation)
plt.bar([0.1373], [0.2720], width=0.01, label="Estimated Value")
plt.plot([exact_value, exact_value], [0,1], 'r--', linewidth=2, label='Exact
                                        Value')
plt.legend()
plt.xticks([0.0, 0.1, 0.2, 0.3, 0.4], size=15)
plt.yticks([0, 0.5, 1], size=15)
plt.title('Estimated Option Price', size=15)
plt.ylabel('Probability', size=15)
plt.ylim((0,1))
plt.grid()
plt.show()
```

REFERENCES

"Qiskit Finance 0.4.1 — qiskit-community.github.io," `https://qiskit-community.github.io/qiskit-finance/`, [Accessed 7-12-2024].

Boyle, P. P., "Options: A Monte Carlo approach," Journal of Financial Economics **4**, 323–338 (1977).

Brassard, G., Hoyer, P., Mosca, M., and Tapp, A., "Quantum Amplitude Amplification and Estimation," Contemporary Mathematics **305** (2002), 10.1090/conm/305/05215.

Dewes, A., Ong, F. R., Schmitt, V., Lauro, R., Boulant, N., Bertet, P., Vion, D., and Esteve, D., "Characterization of a two-transmon processor with individual single-shot qubit readout," Phys. Rev. Lett. **108**, 057002 (2012).

Grinko, D., Gacon, J., Zoufal, C., and Woerner, S., en"Iterative quantum amplitude estimation," Npj Quantum Inf. **7** (2021).

Grover, L.and Rudolph, T., "Creating superpositions that correspond to efficiently integrable probability distributions," (2002), arXiv:quant-ph/0208112.

Hull, J., *Options, futures, and other derivatives*, 6th ed. (Pearson Prentice Hall, Upper Saddle River, NJ [u.a.], 2006).

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., and Gambetta, J. M., "Quantum computing with qiskit," (2024), arXiv:2405.08810 [quant-ph].

Montanaro, A., "Quantum speedup of monte carlo methods," Proceedings of the Royal Society of London A: Mathematical, Physical and Engineering Sciences **471** (2015), 10.1098/rspa.2015.0301.

Nielsen, M. A.and Chuang, I. L., *Quantum Computation and Quantum Information* (Cambridge University Press, 2000).

Stamatopoulos, N., Egger, D. J., Sun, Y., Zoufal, C., Iten, R., Shen, N., and Woerner, S., en"Option pricing using quantum computers," Quantum **4**, 291 (2020).

Suzuki, Y., Uno, S., Raymond, R., Tanaka, T., Onodera, T., and Yamamoto, N., "Amplitude estimation without phase estimation," Quantum Information Processing **19**, 75 (2020).

Temme, K., Bravyi, S., and Gambetta, J. M., "Error mitigation for short-depth quantum circuits," Phys. Rev. Lett. **119**, 180509 (2017).

Woerner, S.and Egger, D. J., en"Quantum risk analysis," Npj Quantum Inf. **5** (2019).

Zoufal, C., Lucchi, A., and Woerner, S., "Quantum generative adversarial networks for learning and loading random distributions," npj Quantum Information **5**, 1–9 (2019).

My response to the questions asked during the talk.

I would like to thank Prof. Colin Benjamin for his constructive comments and insightful questions, which have greatly contributed to a deeper understanding of the concepts explored in this project. Please see below my responses to the questions asked. For the readers' convenience, I have highlighted my responses in blue.

Response to Prof. Colin Benjamin

Prof. Benjamin's Question — "Is it possible to use the QR method, as discussed in class, to approximate the eigenvalues of the Grover operator ($\mathcal{Q}$) as an alternative to the Quantum Phase Estimation (QPE) algorithm?"

Reply: It is theoretically possible to approximate the eigenvalues of the Grover operator $\mathcal{Q}$ using the QR method, but this approach presents several practical limitations, especially in a quantum setting. The classical QR method iteratively decomposes a matrix A into an orthogonal matrix Q and an upper triangular matrix R, converging to a triangular form where the eigenvalues appear along the diagonal. The process is given by:

$$A^{(k)} = Q_k R_k, \quad A^{(k+1)} = R_k Q_k, \tag{1}$$

where $A^{(k+1)}$ eventually converges to an upper triangular matrix, with the eigenvalues of A approximated by the diagonal elements of $A^{(k+1)}$. The convergence depends on the separation of the eigenvalues, with error convergence approximately given by:

$$\text{Error} \approx O\left(\left|\frac{\lambda_j}{\lambda_i}\right|^k\right), \tag{2}$$

where λ_i and λ_j are the largest and second-largest eigenvalues in magnitude, respectively, and k is the number of iterations. The runtime for the classical QR method scales as $O(N^3)$, making it computationally expensive for large matrices and impractical for iterative approximation on quantum hardware [6]. Quantum Phase Estimation (QPE) offers a more efficient alternative for unitary operators, such as $\mathcal{Q}$, by estimating the phase θ in the eigenvalue $e^{i\theta}$ associated with an eigenstate. QPE prepares a superposition, applies controlled unitary operations, and performs the quantum Fourier transform (QFT) to estimate θ with an error rate:

$$|\varphi - \hat{\varphi}| = O\left(\frac{1}{2^m}\right), \tag{3}$$

where m is the number of qubits in the ancilla register, leading to an exponential convergence rate. QPE's runtime scales as $O(\text{poly}(m))$, making it significantly faster and more efficient than the QR method for eigenvalue estimation. QPE's polynomial runtime scales more favorably with precision rather than with the size of the input matrix. For instance if you need $m \sim \log(N)$ qubits to represent the precision accurately, then poly(m) effectively scales as poly(log(N)), which remains sub-linear in N. The paper "Quantum QR Decomposition in the Computational Basis" [3] introduces a quantum QR decomposition algorithm with a runtime of:

$$O\left(\frac{1}{\epsilon^2} N^{2.5}\,\text{polylog}(N)\right), \tag{4}$$

where ϵ is the desired precision. This quantum algorithm improves on the classical runtime, but its reliance on computational basis encoding and updatable quantum memory introduces significant overhead. The iterative quantum QR process amplifies errors due to normalization and rounding at each step. For instance, with the Gram-Schmidt method used in the quantum algorithm, the recursive step for the orthogonal vector $a^{(k+1)}$ is given by:

$$a_{ik}^{(k+1)} = a_{ik}^{(k)} - \sum_{l<k} \langle a_l^{(k)}, a_k^{(k)} \rangle a_{il}^{(k)}, \tag{5}$$

which accumulates errors as each new vector depends on the orthogonality of previous vectors, ultimately leading to instability for high condition numbers. Thus, even with a quantum QR decomposition, QPE remains the preferred choice for eigenvalue estimation on near-term quantum hardware, as it avoids iterative decomposition and provides exponential precision with minimal overhead.

Prof. Benjamin's Question — "What is an affine transformation?"

Reply: An affine transformation is a type of function that maps points from one space to another, preserving points, straight lines, and planes. It can be seen as a linear transformation followed by a translation, and mathematically, it is of the form:

$$f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}, \tag{6}$$

where:

- $\mathbf{x}$ is a point in space represented by a vector,
- A is a linear transformation matrix,
- $\mathbf{b}$ is a translation vector.

The affine transformation preserves collinearity and ratios of distances between points lying on a straight line but does not necessarily preserve angles or lengths. A key feature of affine transformations is that they map parallel lines to parallel lines. Affine transformations maintain the "affine structure" of a space, meaning that the relations between points (such as parallelism and midpoints) remain unchanged, though angles and lengths might not be preserved [5].

In the context of probability distribution loading form the slide, we use an affine transformation to map the index i (which ranges from 0 to $2^n - 1$) to a specified interval $[\text{low}, \text{high}]$. Here, the purpose of the affine transformation is to scale and shift the integer values in $\{0, \ldots, 2^n - 1\}$ into the continuous interval $[\text{low}, \text{high}]$, which represents the range of the log-normal random variable that we are encoding into the quantum state. The affine transformation used here can be written as:

$$i \mapsto \frac{\text{high} - \text{low}}{2^n - 1} \times i + \text{low}. \tag{7}$$

Thus, each integer $i \in \{0, \ldots, 2^n - 1\}$ is mapped to a real value in $[\text{low}, \text{high}]$ through this affine transformation, allowing us to assign each state $|i\rangle_n$ a probability amplitude $\sqrt{p_i}$ corresponding to the values of the log-normal distribution truncated to $[\text{low}, \text{high}]$. This ensures that the quantum state $|\psi\rangle_n$ represents the desired probability distribution accurately over the specified interval [4].

Prof. Benjamin's Question — "What does μ signify in the log-normal probability distribution?"

Reply: In the context of the log-normal probability distribution applied to asset prices, μ represents the mean of the logarithm of the asset price at maturity T. Specifically, for a random variable S_T (the asset price at maturity), $\ln(S_T)$ follows a normal distribution, with mean μ and variance $\sigma^2 T$. The log-normal distribution is commonly used in financial modeling, as it reflects the behavior of asset prices, which cannot fall below zero and tend to exhibit asymmetric growth over time. In this model, the parameter μ is given by:

$$\mu = \left(r - 0.5\sigma^2\right) T + \ln(S_0), \tag{8}$$

where:

- r is the risk-free interest rate, representing the return on an asset with no risk over the time period T.
- σ is the volatility of the asset, quantifying the standard deviation of the asset's returns.
- T is the time to maturity.
- S_0 is the spot price of the asset at the initial time $t = 0$.

The parameter μ plays a central role in determining the expected position of the asset price distribution at maturity. It encapsulates both the deterministic growth component rT (arising from the risk-free rate) and the correction factor $-0.5\sigma^2 T$, which adjusts for the stochastic nature of the asset price due to volatility.

In the Black-Scholes-Merton (BSM) model, μ reflects the "drift" term of the asset price's log-normal distribution, determining the central tendency of S_T. When the volatility σ is high, the uncertainty in S_T increases, but the expected mean (in log-space) is still governed by μ. Thus, μ dictates where the distribution is centered and, implicitly, the expected future price of the asset under the risk-neutral measure. Overall, μ encapsulates both the deterministic growth due to the risk-free rate and a correction for volatility, balancing these factors to define the expected log-return of the asset over the time horizon T [2].

Prof. Benjamin's Question — "What do you mean by price paths?"

Reply: In option pricing, price paths represent potential trajectories of an asset's price over time until the option's maturity. To model these, we often assume the asset follows a stochastic process, such as Geometric Brownian Motion (GBM). Under GBM, the asset price S_t evolves according to the stochastic differential equation:

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t, \tag{9}$$

where r is the risk-free rate, σ is the asset's volatility, and dW_t is a Wiener process.

In Monte Carlo simulations, we discretize this process, generating successive price points $S_{t+\Delta t}$ for small time steps Δt, using:

$$S_{t+\Delta t} = S_t \exp\left(\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\sqrt{\Delta t}\,Z\right), \tag{10}$$

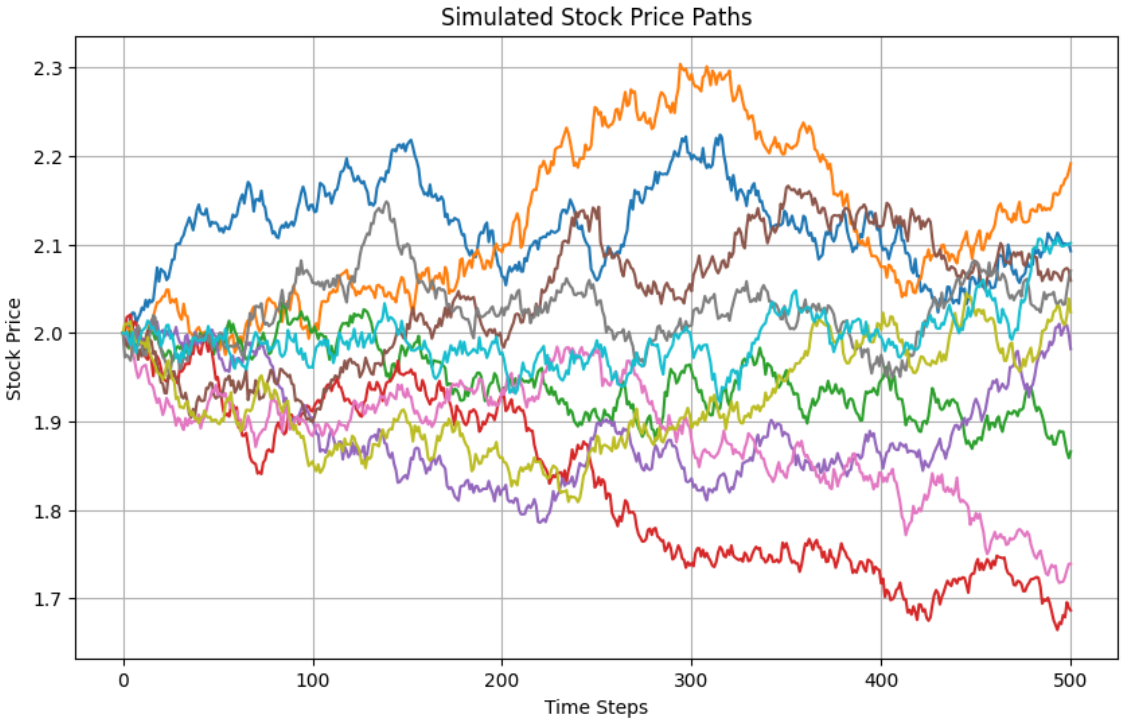


Figure 1: Monte Carlo simulation of possible stock price paths of an European Call Option(plot contains only the first 10 simulations for clarity)

where $Z \sim \mathcal{N}(0,1)$ introduces randomness. Starting from S_0, we simulate multiple price paths until maturity T, representing possible futures for the asset. Each path gives a final price S_T, which we use to calculate the option's payoff. For a European call, this is $\max(S_T - K, 0)$, where K is the strike price. We then average the payoffs across all paths and discount to the present using e^{-rT} to get the option price:

$$\text{Option Price} = e^{-rT}\,\mathbb{E}[\text{Payoff}]. \tag{11}$$

Prof. Benjamin's Question — "In a prior slide, you mentioned that, at this stage of development, quantum computing does not necessarily offer superior computational power over classical computing for solving problems that are considered 'computationally easy' or tractable. However, for computationally hard problems, quantum computing can potentially demonstrate a significant advantage. Could you please elaborate on this point and discuss the specific contexts in which quantum computing may offer such a computational advantage?"

Reply: In the current stage of quantum computing, classical computers still dominate for many types of problems, especially those that are considered "computationally easy" or tractable on classical machines. These problems generally belong to the class P, meaning they can be solved in polynomial time with respect to the input size, using classical algorithms. For instance, sorting, simple arithmetic, and certain types of optimization problems fall within this range and are efficiently handled by classical algorithms. Classical computing is particularly effective when polynomial-time algorithms are available, and for these tractable tasks, the overhead of quantum systems—such as error correction, coherence maintenance,

and qubit quality—often negates any potential speedup. From the paper I was tasked to present on [4], I quote the following- "Options that are path-independent and rely on a single asset are the easiest to price, and in most cases numerical calculation is straightforward and would likely not benefit by the use of a quantum computer. Path-independent options on multiple assets are only slightly harder to price since more than one asset is now involved and the probability distributions must account for correlations between the assets, but usually these can be priced quite efficiently on classical computers as well. Path-dependent options on the other hand are significantly harder to price than path-independent options since they require an often expensive payoff calculation at multiple time points on each path, therefore minimizing the number of paths required for this step would lead to a significant benefit in the pricing process. It is this last case where we envision the largest impact of quantum computing."

Quantum optimization algorithms, though not universally faster for all problem types, introduce new tools that may enhance performance for certain instances. This highlights the importance of understanding the theoretical and empirical advantages of quantum methods and exploring synergies with classical algorithms to potentially enhance optimization capabilities. "In practice, it is often necessary to rely on optimization heuristics, in which the performance for given problem instances is not known upfront. This makes comparing quantum and classical algorithms rather tricky, given the limitations of what can be run on today's quantum hardware" [1].

For example, in integer factorization and discrete logarithm problems, classical algorithms, such as the general number field sieve, operate in sub-exponential time $O\left(\exp\left((64/9)^{1/3}(\log N)^{1/3}(\log\log N)^{2/3}\right)\right)$, while Shor's algorithm [7] runs in polynomial time, $O((\log n)^2 \log\log n)$, providing an exponential speedup in cryptographic contexts. In unstructured search, Grover's algorithm offers a quadratic improvement, reducing the search time from $O(N)$ to $O(\sqrt{N})$, which is valuable for search-related problems but does not surpass exponential bounds. Quantum algorithms like the Quantum Approximate Optimization Algorithm (QAOA) and Variational Quantum Eigensolver (VQE) offer potential in combinatorial optimization, although they currently lack guaranteed exponential speedups. QAOA, for example, can operate in polynomial time relative to the depth of the quantum circuit and the problem size, potentially providing more efficient approximation solutions for specific instances than classical heuristics. These quantum algorithms introduce new solution landscapes by utilizing quantum superposition and entanglement, allowing exploration of exponentially large state spaces in a way that classical algorithms cannot. Consequently, quantum runtimes, especially in cases of high-dimensional or combinatorially complex problems such as material simulations, portfolio optimization, or machine learning, offer a promising edge where classical runtimes scale prohibitively [1].

In the context of the paper assigned to me, the authors show a quadratic speed up over classical Monte Carlo methods using Amplitude Estimation to price options. In the figure below, the authors show the comparison between the two to price a Basket option.

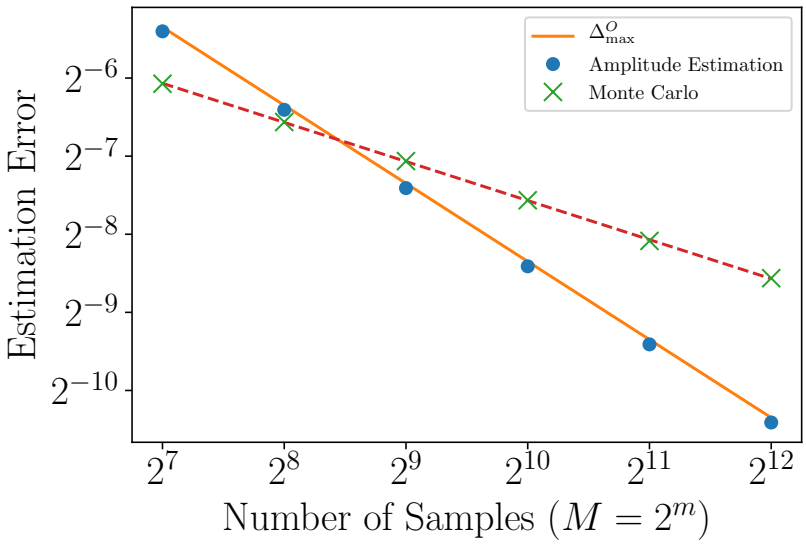


Figure 2: Comparison of estimation errors between Amplitude Estimation and Monte Carlo at an 81% confidence interval for a basket option with three identical, equally weighted assets. Parameters include a strike price of $K = 2.0$ and asset correlations $\rho_{12} = \rho_{13} = \rho_{23} = 0.8$. Amplitude Estimation shows $\mathcal{O}(M^{-1})$ convergence, while Monte Carlo errors, calculated over 10,000 simulations for each sample 2^m, exhibit the expected $\mathcal{O}(M^{-1/2})$ scaling, with a linear fit (red dashed line) [4].

References

[1] Amira Abbas et al. "Challenges and opportunities in quantum optimization". en. In: *Nat. Rev. Phys.* (Oct. 2024).

[2] J. Hull. *Options, Futures & Other Derivatives*. Prentice Hall International editions v. 1. Prentice Hall, 2000. ISBN: 9780130158222. URL: https://books.google.co.in/books?id=6FCcSAAACAAJ.

[3] Guangsheng Ma, Hongbo Li, and Jiman Zhao. "Quantum QR decomposition in the computational basis". en. In: *Quantum Inf. Process.* 19.8 (Aug. 2020).

[4] Nikitas Stamatopoulos et al. "Option Pricing using Quantum Computers". en. In: *Quantum* 4.291 (July 2020), p. 291.

[5] Wikipedia contributors. *Affine transformation — Wikipedia, The Free Encyclopedia*. [Online; accessed 14-November-2024]. 2024. URL: https://en.wikipedia.org/w/index.php?title=Affine_transformation&oldid=1256188832.

[6] Wikipedia contributors. *QR algorithm — Wikipedia, The Free Encyclopedia*. [Online; accessed 14-November-2024]. 2024. URL: https://en.wikipedia.org/w/index.php?title=QR_algorithm&oldid=1254105269.

[7] Wikipedia contributors. *Shor's algorithm — Wikipedia, The Free Encyclopedia*. [Online; accessed 14-November-2024]. 2024. URL: https://en.wikipedia.org/w/index.php?title=Shor%27s_algorithm&oldid=1256780837.